Reviewer Pack — Minimal Order of Tropical Symplectic Leaves and Resolution P…

Entry 452f0bf61796 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 06:55:25 UTC

Minimal Order of Tropical Symplectic Leaves and Resolution Proof Width Inequality

Entry ID: `452f0bf61796` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 06:55:25 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every instance φ of SAT, the minimal order of tropical symplectic leaves (ost(Lφ)) in its associated matroid Lφ is linearly correlated with its resolution proof width w(φ), such that ost(Lφ) = Θ(w(φ)).

Rationale (proposer's reasoning):

> Tropical symplectic geometry provides a rich framework for understanding the geometric properties of combinatorial objects, including matroids. The conjecture posits that the order of these leaves is an informative invariant for resolution proof complexity, potentially revealing underlying geometric structures in SAT instances.

Taxonomy category: `TROPICAL_GEOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3b50ac9bf176ca14`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between ost(Lφ) and w(φ) for 30 random seeds is ≥ 0.95, with no seed producing a correlation coefficient < 0.9.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND resolution proof complexity - minimal order tropical symplectic leaves AND resolution proof width - θ(w(φ)) AND matroid Lφ

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        clauses = []
        for _ in range(m):
            clause = random.sample(range(1, n+1), 3)
            clauses.append(clause)
        return clauses

    def resolution_width(cnf):
        queue = cnf[:]
        seen = set()
        while queue:
            literal = queue.pop()
            if literal in seen or -literal in seen:
                continue
            seen.add(literal)
            for clause in cnf:
                if literal in clause:
                    new_clause = [l for l in clause if l != literal]
                    if not new_clause:
                        return len(queue) + 1
                    queue.append(new_clause)
        return float('inf')

    def matroid_rank(cnf):
        rank = 0
        seen = set()
        for clause in cnf:
            for literal in clause:
```

```
                if literal not in seen:
                    seen.add(literal)
                    rank += 1
        return rank

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m = random.randint(2 * n, 3 * n)
        cnf = generate_cnf(n, m)
        ost_L_phi = matroid_rank(cnf)
        w_phi = resolution_width(cnf)
        results.append((ost_L_phi, w_phi))

    if len(results) < 30:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "not_enough_instances"
        }

    ost_L_phi_values = [r[0] for r in results]
    w_phi_values = [r[1] for r in results]

    mean_ost_L_phi = sum(ost_L_phi_values) / len(ost_L_phi_values)
    mean_w_phi = sum(w_phi_values) / len(w_phi_values)

    covariance = sum((ost_L_phi_values[i] - mean_ost_L_phi) * (w_phi_values[i] - mean_w_phi) for i
    variance_ost_L_phi = sum((ost_L_phi_values[i] - mean_ost_L_phi) ** 2 for i in range(len(result
    variance_w_phi = sum((w_phi_values[i] - mean_w_phi) ** 2 for i in range(len(results))) / len(r

    correlation_coefficient = covariance / (math.sqrt(variance_ost_L_phi) * math.sqrt(variance_w_p

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": 30,
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.95,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient<0.95\" first_failing_se
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4589cccf.py", line 101, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4589cccf.py", line 61, in run_trial
    w_phi = resolution_width(cnf)
            ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4589cccf.py", line 33, in resolution_wi
    if literal in seen or -literal in seen:
       ^^^^^^^^^^^^^^^
TypeError: unhashable type: 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete the required calculations to determine the Pearson correlation coefficient between ost(Lφ) and w(φ). As a result, we cannot confirm whether the support condition for the conjecture has been met. | next: Re-run the test with proper error handling to ensure that it completes without crashing. If the test passes, re-evaluate the results based on the pre-registered criteria.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13354
2	preregistration	ollama_remote	glm4:latest	0	0	9061
3	novelty	ollama_remote	glm4:latest	0	0	9964
4	novelty	ollama_remote	glm4:latest	0	0	8753
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31930
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13131
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12931
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11806
9	judge	ollama_remote	glm4:latest	0	0	28445

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 139377 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/452f0bf61796.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/452f0bf61796.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/452f0bf61796.tar.gz` (if generated)